

Generated: December 11, 2025
Project: Busy Beans Coffee Distribution Platform
Document: ERD and Flow Diagrams

Busy Beans Backend - ERD and Flow Diagrams

Generated: December 11, 2025
Project: Busy Beans Coffee Distribution Platform



Complete Entity Relationship Diagram (ERD)

Full Database Schema with All Relationships

```
erDiagram
    %% User Management Entities
    account ||--o{ employee : "employs"
    account ||--o{ order : "manages"

    user ||--o{ order : "places"
    user ||--o{ address : "has"
    user ||--o{ billingAddress : "has"
    user ||--o{ orderFrequency : "subscribes"
    user ||--o{ userDiscount : "receives"
    user ||--o{ deviceToken : "has"
    user ||--o{ qboCustomerMap : "mapped_to"
    user }o--|| salesRep : "managed_by"

    salesRep ||--o{ user : "manages"
    salesRep ||--o{ order : "facilitates"
    salesRep ||--o{ partnerOrder : "places"
    salesRep ||--o{ employee : "employs"
    salesRep ||--o{ address : "has"
    salesRep ||--o{ billingAddress : "has"
    salesRep ||--o{ orderFrequency : "schedules"
    salesRep ||--o{ deviceToken : "has"
    salesRep ||--o{ salesFromPartners : "generates"
    salesRep ||--o{ transfersToSalesRep : "receives"
    salesRep ||--o{ qboCredentials : "has"
    salesRep ||--o{ qboToken : "has"
    salesRep ||--o{ qboCustomerMap : "mapped_to"
    salesRep ||--|| stateInSystem : "territory"

    employee ||--o{ employeePermission : "has"
    employee }o--|| salesRep : "works_for"
    employee }o--|| account : "works_for"

    permission ||--o{ employeePermission : "granted_in"

    %% Product & Catalog Entities
    category ||--o{ product : "contains"

    product ||--o{ item : "ordered_as"
    product ||--o{ partnerOrderItem : "ordered_as"
    product ||--o{ userDiscount : "discounted_in"
    product ||--o{ skuSupplier : "supplied_by"
```

```
supplier ||--o{ skuSupplier : "supplies"

coffeeMachine ||--o{ subscription : "subscribed_to"
coffeeMachine ||--o{ Lead : "interested_in"

%% Order Management Entities
order ||--o{ item : "contains"
order ||--o{ orderHistory : "tracks"
order ||--|| orderFrequency : "scheduled_by"
order ||--|| chequeDetail : "paid_by"
order ||--|| transfersToSalesRep : "generates"
order }o--|| user : "placed_by"
order }o--|| salesRep : "facilitated_by"

partnerOrder ||--o{ partnerOrderItem : "contains"
partnerOrder ||--o{ orderHistory : "tracks"
partnerOrder ||--|| chequeDetail : "paid_by"
partnerOrder }o--|| salesRep : "placed_by"

item }o--|| product : "references"
item }o--|| order : "belongs_to"

partnerOrderItem }o--|| product : "references"
partnerOrderItem }o--|| partnerOrder : "belongs_to"

orderFrequency }o--|| user : "for_user"
orderFrequency }o--|| salesRep : "managed_by"
orderFrequency }o--|| order : "creates"

orderHistory }o--|| statuses : "has_status"
orderHistory }o--|| order : "tracks"
orderHistory }o--|| partnerOrder : "tracks"

statuses ||--o{ orderHistory : "used_in"

%% Lead Management Entities
Lead ||--o{ LeadLog : "has_logs"
Lead }o--|| salesRep : "assigned_to"
Lead }o--|| employee : "assigned_to"
Lead }o--|| coffeeMachine : "interested_in"

LeadLog }o--|| Lead : "logs_for"

%% Subscription Entities
subscription ||--o{ subscriptionAddon : "has_addons"
subscription }o--|| coffeeMachine : "for_machine"

addon ||--o{ subscriptionAddon : "in_subscriptions"
```

```
subscriptionAddon }o--|| subscription : "belongs_to"
subscriptionAddon }o--|| addon : "references"

%% Payment & Financial Entities
chequeDetail }o--|| order : "pays"
chequeDetail }o--|| partnerOrder : "pays"

transfersToSalesRep }o--|| order : "from_order"
transfersToSalesRep }o--|| salesRep : "to_salesrep"

salesFromPartners }o--|| salesRep : "from_salesrep"

%% QuickBooks Integration Entities
qboToken }o--|| salesRep : "for_salesrep"

qboCredentials }o--|| salesRep : "for_salesrep"

qboCustomerMap }o--|| user : "maps_user"
qboCustomerMap }o--|| salesRep : "maps_salesrep"

%% Address & Location Entities
countryInSystem ||--o{ stateInSystem : "contains"
stateInSystem ||--o{ cityInSystem : "contains"
stateInSystem }o--|| salesRep : "territory_for"

territory ||--o{ salesRep : "assigned_to"

%% Shipping Entities
shippingCompanies ||--o{ order : "ships"

%% System Entities
deviceToken }o--|| user : "for_user"
deviceToken }o--|| salesRep : "for_salesrep"

pushNotifications ||--o{ deviceToken : "sent_to"
```



Complete Order Processing Flow

Detailed Order Lifecycle

flowchart TD

Start([Customer Places Order]) --> Validate{Validate Order Data}

Validate -->|Invalid| Error1[Return Error]

Validate -->|Valid| Calculate[Calculate Pricing]

Calculate --> CalcItems[Calculate Items Price]

CalcItems --> CalcDiscount[Apply Discounts]

CalcDiscount --> CalcShipping[Calculate Shipping Charges]

CalcShipping --> CalcVAT[Add VAT]

CalcShipping --> CalcTotal[Calculate Total Bill]

CalcTotal --> CreateOrder[Create Order Record]

CreateOrder --> CreateItems[Create Order Items]

CreateItems --> CreateHistory[Create Order History Entry]

CreateHistory --> GenInvoice[Generate Invoice Number]

GenInvoice --> Payment{Payment Method?}

Payment -->|Stripe| StripeFlow[Stripe Payment Flow]

Payment -->|Invoice| InvoiceFlow[Invoice Flow]

Payment -->|Check| CheckFlow[Check Flow]

StripeFlow --> CreateIntent[Create Payment Intent]

CreateIntent --> Charge[Charge Customer]

Charge -->|Success| UpdatePaid[Update Payment Status = done]

Charge -->|Failed| UpdatePending[Update Payment Status = pending]

InvoiceFlow --> SetTerms[Set Invoice Terms Net 30/60/90]

SetTerms --> SetReminder[Set Invoice Reminder Date]

SetReminder --> UpdatePending

CheckFlow --> RecordCheck[Record Check Details]

RecordCheck --> UpdatePending

UpdatePaid --> SendEmail1[Send Order Confirmation Email]

UpdatePending --> SendEmail1

SendEmail1 --> AssignSupplier{Assign Supplier?}

AssignSupplier -->|Yes| NotifySupplier[Notify Supplier]

AssignSupplier -->|No| WaitAssignment[Wait for Assignment]

NotifySupplier --> SupplierAck{Supplier Acknowledges?}

SupplierAck -->|Yes| UpdateAck[Update Order Status]

```
SupplierAck -->|No| WaitAck[Wait for Acknowledgment]

UpdateAck --> Dispatch{Order Dispatched?}
Dispatch -->|Yes| AddTracking[Add Tracking Number]
AddTracking --> UpdateDispatch[Update Order Status]
UpdateDispatch --> SendDispatchEmail[Send Dispatch Email]
SendDispatchEmail --> Delivery{Order Delivered?}

Delivery -->|Yes| UpdateDelivered[Update Order Status = Delivered]
Delivery -->|No| WaitDelivery[Wait for Delivery]

UpdateDelivered --> SyncQBO{QuickBooks Connected?}
SyncQBO -->|Yes| SyncInvoice[Sync Invoice to QBO]
SyncQBO -->|No| SkipQBO[Skip QBO Sync]

SyncInvoice --> CalcCommission{Has Sales Rep?}
CalcCommission -->|Yes| CalcComm[Calculate Commission]
CalcCommission -->|No| SkipCommission[Skip Commission]

CalcComm --> RecordTransfer[Record Commission Transfer]
RecordTransfer --> ProcessPayout{Process Payout?}
ProcessPayout -->|Stripe| ImmediatePayout[Immediate Stripe Transfer]
ProcessPayout -->|Bank| SchedulePullout[Schedule Bank Pullout]

ImmediatePayout --> SyncPayment[Sync Payment to QBO]
SchedulePullout --> SyncPayment
SkipCommission --> SyncPayment
SkipQBO --> End([Order Complete])
SyncPayment --> End

Error1 --> End
WaitAssignment --> AssignSupplier
WaitAck --> SupplierAck
WaitDelivery --> Delivery
```



Payment Processing Flow

Multi-Payment Method Flow

```
sequenceDiagram
    participant C as Customer
    participant API as API Server
    participant DB as Database
    participant S as Stripe
    participant QBO as QuickBooks
    participant E as Email Service

    Note over C,E: Payment Method: Stripe Card
    C->>API: Place Order (paymentMethod: card)
    API->>DB: Create Order
    API->>S: Create Payment Intent
    S-->>API: Payment Intent Created
    API->>C: Return Client Secret

    C->>S: Confirm Payment (Frontend)
    S->>API: Webhook: payment_intent.succeeded
    API->>DB: Update paymentStatus = done
    API->>DB: Store paymentIntentId
    API->>E: Send Payment Confirmation Email
    API->>QBO: Sync Payment (if QBO connected)

    Note over C,E: Payment Method: Invoice (Net Terms)
    C->>API: Place Order (paymentMethod: invoice)
    API->>DB: Create Order
    API->>DB: Set paymentStatus = pending
    API->>DB: Set invoiceDate, termDays, invoiceReminder
    API->>E: Send Invoice Email
    API->>QBO: Create Invoice in QBO

    Note over C,E: Payment Received Later
    API->>DB: Update paymentStatus = done
    API->>DB: Set invoicePaidDate
    API->>S: Record Payment (if needed)
    API->>QBO: Record Payment in QBO
    API->>E: Send Payment Confirmation Email

    Note over C,E: Payment Method: Check
    C->>API: Place Order (paymentMethod: check)
    API->>DB: Create Order
    API->>DB: Create Cheque Record
    API->>DB: Set paymentStatus = pending
    API->>E: Send Order Confirmation
```

```
Note over C,E: Check Cleared
API->>>DB: Update paymentStatus = done
API->>>DB: Update Cheque Status
API->>>QBO: Record Payment in QBO
API->>>E: Send Payment Confirmation Email

Note over C,E: Partner Order with Commission
C->>>API: Place Order (with salesRep)
API->>>DB: Calculate Commission
API->>>DB: Create Order
API->>>DB: Record Commission in transfersToSalesRep
API->>>S: Create Payment Intent (with Connect)
S->>>API: Split Payment (Admin + Partner)
API->>>DB: Update adminReceivableAmount
API->>>DB: Update localPartnerCommission
API->>>QBO: Sync to Admin QBO
API->>>QBO: Sync to Partner QBO
```



Lead Management Flow

Complete Lead Pipeline

```
stateDiagram-v2
    [*] --> NewEnquiry: Lead Created

    NewEnquiry --> Contacted: First Contact Made
    NewEnquiry --> LOST: Not Interested

    Contacted --> Quoted: Quotation Sent
    Contacted --> Demo: Site Visit Scheduled
    Contacted --> LOST: No Response

    Quoted --> Demo: Demo Scheduled
    Quoted --> Negotiation: Customer Interested
    Quoted --> WON: Quick Win
    Quoted --> LOST: Price Too High

    Demo --> Negotiation: Discussion Started
    Demo --> WON: Immediate Purchase
    Demo --> LOST: Chose Competitor

    Negotiation --> Nurture: Need More Time
    Negotiation --> WON: Deal Closed
    Negotiation --> LOST: Budget Issues

    Nurture --> Contacted: Follow-up
    Nurture --> Negotiation: Ready to Proceed
    Nurture --> LOST: Lost Interest

    WON --> [*]: Convert to Customer
    LOST --> [*]: Archive Lead

    note right of NewEnquiry
        Lead Source:
        - Instagram
        - Website
        - Referral
        - Cold Call
        - WhatsApp
    end note

    note right of Quoted
        Actions:
        - Send Quotation Email
        - Record Amount
```

```
    - Set Follow-up Date  
end note
```

```
note right of Demo  
  Actions:  
    - Schedule Site Visit  
    - Add Visit Notes  
    - Record Machine Interest  
end note
```

```
note right of WON  
  Actions:  
    - Create User Account  
    - Assign Sales Rep  
    - Create First Order  
end note
```

Lead Assignment and Activity Flow

```
flowchart TD
    LeadCreated[New Lead Created] --> LogActivity1[Log: Lead Created]
    LogActivity1 --> Assign{Assign Lead?}

    Assign -->|To Sales Rep| AssignSR[Assign to Sales Rep]
    Assign -->|To Employee| AssignEmp[Assign to Employee]
    Assign -->|Unassigned| Unassigned[Keep Unassigned]

    AssignSR --> LogActivity2[Log: Assigned to Sales Rep]
    AssignEmp --> LogActivity3[Log: Assigned to Employee]

    LogActivity2 --> FollowUp{Follow-up Needed?}
    LogActivity3 --> FollowUp
    Unassigned --> FollowUp

    FollowUp -->|Yes| ScheduleFollowUp[Schedule Follow-up Date]
    ScheduleFollowUp --> LogActivity4[Log: Follow-up Scheduled]
    FollowUp -->|No| WaitAction[Wait for Action]

    WaitAction --> Action{Action Taken?}
    Action -->|Send Quotation| SendQuotation[Send Quotation]
    Action -->|Schedule Visit| ScheduleVisit[Schedule Site Visit]
    Action -->|Update Status| UpdateStatus[Update Lead Status]
    Action -->|Add Note| AddNote[Add Note/Feedback]

    SendQuotation --> LogActivity5[Log: Quotation Sent]
    LogActivity5 --> UpdateQuotation[Update: quotationSent = true]
    UpdateQuotation --> UpdateStatusLead[Update Status: Quoted]

    ScheduleVisit --> LogActivity6[Log: Site Visit Scheduled]
    LogActivity6 --> UpdateVisit[Update: siteVisitScheduled = true]
    UpdateVisit --> UpdateStatusVisit[Update Status: Demo/Scheduled]

    UpdateStatus --> LogActivity7[Log: Status Changed]
    AddNote --> LogActivity8[Log: Note Added]

    UpdateStatusLead --> WonLost{Won or Lost?}
    UpdateStatusVisit --> WonLost
    LogActivity7 --> WonLost
    LogActivity8 --> WonLost

    WonLost -->|WON| MarkWon[Mark as WON]
    WonLost -->|LOST| MarkLost[Mark as LOST]

    MarkWon --> LogActivity9[Log: Lead Won]
    LogActivity9 --> CreateCustomer[Create Customer Account]
```

```
CreateCustomer --> ConvertComplete[Conversion Complete]

MarkLost --> LogActivity10[Log: Lead Lost]
LogActivity10 --> RecordReason[Record Lost Reason]
RecordReason --> RecordFeedback[Record Customer Feedback]
RecordFeedback --> ArchiveLead[Archive Lead]

ConvertComplete --> [*]
ArchiveLead --> [*]
```



QuickBooks Online Integration Flow

Complete QBO Sync Workflow

flowchart TD

```
Start([Order/Payment Event]) --> CheckConnection{QBO Connected?}
CheckConnection -->|No| Skip[Skip QBO Sync]
CheckConnection -->|Yes| CheckToken{Token Valid?}

CheckToken -->|Expired| RefreshToken[Refresh Access Token]
CheckToken -->|Valid| GetCredentials[Get QBO Credentials]
RefreshToken --> GetCredentials

GetCredentials --> CheckCustomer{Customer Synced?}

CheckCustomer -->|No| CreateCustomer[Create QBO Customer]
CreateCustomer --> StoreCustomerId[Store qboCustomerId]
StoreCustomerId --> CheckCustomer2{Customer Synced?}

CheckCustomer -->|Yes| GetCustomerId[Get qboCustomerId]
CheckCustomer2 -->|Yes| GetCustomerId

GetCustomerId --> SyncType{What to Sync?}

SyncType -->|Invoice| SyncInvoice[Sync Invoice]
SyncType -->|Payment| SyncPayment[Sync Payment]

SyncInvoice --> CheckInvoice{Invoice Exists?}
CheckInvoice -->|No| CreateInvoice[Create QBO Invoice]
CheckInvoice -->|Yes| UpdateInvoice[Update QBO Invoice]

CreateInvoice --> StoreInvoiceId[Store quickBooksInvoiceId]
UpdateInvoice --> StoreInvoiceId
StoreInvoiceId --> CheckPartner{Partner Order?}

CheckPartner -->|Yes| SyncPartnerInvoice[Sync to Partner QBO]
CheckPartner -->|No| UpdateSyncTime[Update qboLastSync]

SyncPartnerInvoice --> StorePartnerInvoiceId[Store quickBooksInvoiceIdPartner]
StorePartnerInvoiceId --> UpdateSyncTime

SyncPayment --> CheckPayment{Payment Synced?}
CheckPayment -->|No| CreatePayment[Create QBO Payment]
CheckPayment -->|Yes| SkipPayment[Skip Payment Sync]

CreatePayment --> StorePaymentId[Store quickBooksPaymentId]
StorePaymentId --> CheckPartnerPayment{Partner Payment?}
```

```
CheckPartnerPayment -->|Yes| SyncPartnerPayment[Sync to Partner QBO]
CheckPartnerPayment -->|No| UpdatePaymentSync[Update paymentSyncedToQBO = true]

SyncPartnerPayment --> StorePartnerPaymentId[Store quickBooksPaymentIdPartner]
StorePartnerPaymentId --> UpdatePaymentSync

UpdateSyncTime --> Success[Sync Success]
UpdatePaymentSync --> Success
SkipPayment --> Success
Skip --> End([End])
Success --> End

CreateCustomer -->|Error| HandleError[Handle QBO Error]
CreateInvoice -->|Error| HandleError
UpdateInvoice -->|Error| HandleError
CreatePayment -->|Error| HandleError

HandleError --> LogError[Log Error in qboSyncError]
LogError --> SetErrorStatus[Set qboSyncStatus = error]
SetErrorStatus --> End
```

Dual QBO Sync (Admin + Partner)

```
sequenceDiagram
    participant Order as Order System
    participant AdminQBO as Admin QBO
    participant PartnerQBO as Partner QBO
    participant DB as Database

    Note over Order,DB: Customer Order with Sales Rep

    Order->>DB: Order Created
    Order->>AdminQBO: Sync Customer
    AdminQBO-->>Order: Customer ID
    Order->>DB: Store qboCustomerId

    Order->>AdminQBO: Create Invoice
    AdminQBO-->>Order: Invoice ID
    Order->>DB: Store quickBooksInvoiceId
    Order->>DB: Store adminRealmId

    Order->>PartnerQBO: Sync Customer (Partner Realm)
    PartnerQBO-->>Order: Customer ID
    Order->>DB: Store qboCustomerIdForPartner

    Order->>PartnerQBO: Create Invoice
    PartnerQBO-->>Order: Invoice ID
    Order->>DB: Store quickBooksInvoiceIdPartner
    Order->>DB: Store partnerRealmId

    Note over Order,DB: Payment Received

    Order->>AdminQBO: Record Payment
    AdminQBO-->>Order: Payment ID
    Order->>DB: Store quickBooksPaymentId

    Order->>PartnerQBO: Record Payment
    PartnerQBO-->>Order: Payment ID
    Order->>DB: Store quickBooksPaymentIdPartner
    Order->>DB: Set paymentSyncedToQBO = true
```



Recurring Order Automation Flow

Automatic Order Creation

flowchart TD

```
Start([Lambda Function Triggered]) --> FetchFrequencies[Fetch Active Order Frequencies]
FetchFrequencies --> Loop{For Each Frequency}
```

```
Loop -->|Next| CheckDate{Next Order Date <= Today?}
CheckDate -->|No| Skip[Skip - Not Due Yet]
CheckDate -->|Yes| GetOrderData[Get Last Order Data]
```

```
GetOrderData --> CreateOrder[Create New Order]
CreateOrder --> CopyItems[Copy Order Items]
CopyItems --> CalculatePrice[Calculate Pricing]
CalculatePrice --> SetFrequency[Set Order Frequency]
SetFrequency --> CreateHistory[Create Order History]
```

```
CreateHistory --> CalculateNext[Calculate Next Order Date]
CalculateNext --> UpdateFrequency[Update Frequency Record]
UpdateFrequency --> SetNextDate[Set nextOrderDate]
```

```
SetNextDate --> CheckStatus{Status Active?}
CheckStatus -->|Yes| KeepActive[Keep Status Active]
CheckStatus -->|No| MarkInactive[Mark as Inactive]
```

```
KeepActive --> SendNotification[Send Notification to Customer]
MarkInactive --> SkipNotification[Skip Notification]
```

```
SendNotification --> ProcessPayment{Payment Method?}
SkipNotification --> NextFrequency[Next Frequency]
```

```
ProcessPayment -->|Stripe| AutoCharge[Auto Charge Customer]
ProcessPayment -->|Invoice| CreateInvoice[Create Invoice]
ProcessPayment -->|Manual| WaitPayment[Wait for Payment]
```

```
AutoCharge -->|Success| UpdatePaid[Update Payment Status]
AutoCharge -->|Failed| UpdatePending[Update Payment Status Pending]
CreateInvoice --> UpdatePending
WaitPayment --> UpdatePending
```

```
UpdatePaid --> NextFrequency
UpdatePending --> NextFrequency
Skip --> NextFrequency
```

```
NextFrequency --> Loop  
Loop -->|Done| End([All Frequencies Processed])
```



Commission Calculation and Payout Flow

Sales Rep Commission Processing

flowchart TD

```
OrderPlaced[Order Placed] --> CheckSalesRep{Has Sales Rep?}
CheckSalesRep -->|No| NoCommission[No Commission]
CheckSalesRep -->|Yes| CheckPartnerType{Partner Type?}

CheckPartnerType -->|Direct Partner| DirectCommission[Commission = Full Order Amount]
CheckPartnerType -->|Dropship Partner| DropshipCommission[Commission = Price - Wholesal

DirectCommission --> CalculateTotal[Calculate Total Commission]
DropshipCommission --> CalculateTotal

CalculateTotal --> CalculateFees[Calculate Stripe Fees]
CalculateFees --> ProportionalFee[Calculate Proportional Fee]
ProportionalFee --> DeductFees[Deduct Fees from Commission]

DeductFees --> NetCommission[Net Commission Amount]
NetCommission --> RecordTransfer[Record in transfersToSalesRep]

RecordTransfer --> StoreCommission[Store localPatnerCommission]
StoreCommission --> CalculateAdmin[Calculate Admin Receivable]
CalculateAdmin --> StoreAdmin[Store adminReceivableAmount]

StoreAdmin --> CheckPayment{Payment Method?}

CheckPayment -->|Stripe Card| StripePayout[Stripe Connect Payout]
CheckPayment -->|Bank Account| BankPayout[Bank Account Pullout]
CheckPayment -->|Invoice| InvoicePayout[Invoice - Defer Payout]

StripePayout --> CreateTransfer[Create Stripe Transfer]
CreateTransfer -->|Success| UpdatePaid[Update adminReceivableStatus = true]
CreateTransfer -->|Failed| UpdatePending[Update adminReceivableStatus = false]

BankPayout --> SchedulePullout[Schedule Pullout Date]
SchedulePullout --> StorePulloutDate[Store pulloutDate]
StorePulloutDate --> LambdaTrigger[Lambda Function Triggered]

LambdaTrigger --> ProcessPullout[Process Bank Pullout]
ProcessPullout --> CreatePulloutIntent[Create Pullout Intent]
CreatePulloutIntent -->|Success| UpdatePaid
CreatePulloutIntent -->|Failed| RetryPullout[Retry Later]

InvoicePayout --> WaitPayment[Wait for Payment]
WaitPayment --> PaymentReceived{Payment Received?}
```

```
PaymentReceived -->|Yes| ProcessPayout[Process Payout]
PaymentReceived -->|No| WaitReminder[Wait for Reminder]

ProcessPayout --> UpdatePaid
UpdatePaid --> SyncQBO[Sync to QuickBooks]
SyncQBO --> SyncAdminQBO[Sync to Admin QBO]
SyncAdminQBO --> SyncPartnerQBO[Sync to Partner QBO]

SyncPartnerQBO --> CommissionComplete[Commission Paid]
UpdatePending --> CommissionPending[Commission Pending]
NoCommission --> NoCommissionEnd[No Commission]

CommissionComplete --> [*]
CommissionPending --> [*]
NoCommissionEnd --> [*]
RetryPullout --> [*]
WaitReminder --> [*]
```



Subscription Management Flow

Coffee Machine Subscription Lifecycle

```
stateDiagram-v2
    [*] --> SelectMachine: Customer Selects Machine
    SelectMachine --> SelectAddons: Choose Add-ons
    SelectAddons --> CalculatePrice: Calculate Total Price

    CalculatePrice --> CreateStripePrice: Create Stripe Price
    CreateStripePrice --> CreateSubscription: Create Stripe Subscription
    CreateSubscription --> StatusIncomplete: Subscription Created

    StatusIncomplete --> StatusTrialing: Trial Started
    StatusIncomplete --> StatusActive: Payment Succeeded
    StatusIncomplete --> StatusCanceled: Cancelled

    StatusTrialing --> StatusActive: Trial Ended
    StatusTrialing --> StatusCanceled: Cancelled During Trial

    StatusActive --> StatusPastDue: Payment Failed
    StatusActive --> StatusCanceled: Customer Cancelled
    StatusActive --> StatusActive: Recurring Payment

    StatusPastDue --> StatusActive: Payment Retried Success
    StatusPastDue --> StatusCanceled: Max Retries Failed

    StatusCanceled --> [*]: Subscription Ended
    StatusActive --> [*]: Ongoing Subscription

    note right of SelectMachine
        Machine Selection:
        - Browse Machines
        - View Specifications
        - Select Machine
    end note

    note right of SelectAddons
        Add-on Options:
        - Maintenance Package
        - Extended Warranty
        - Training Sessions
        - Additional Equipment
    end note

    note right of CalculatePrice
        Price Calculation:
```

```
- Machine Base Price
- Add-on Prices
- Dynamic Stripe Price
end note
```

```
note right of StatusActive
  Active Subscription:
    - Recurring Billing
    - Access to Services
    - Support Included
end note
```



Authentication and Authorization Flow

Complete Auth Flow with Redis

```
sequenceDiagram
    participant Client
    participant API
    participant DB
    participant JWT
    participant Redis
    participant Bcrypt

    Note over Client,Bcrypt: Login Flow
    Client->>API: POST /login (email, password)
    API->>DB: Find User by Email
    DB-->>API: User Record

    API->>Bcrypt: Compare Password
    Bcrypt-->>API: Password Valid/Invalid

    alt Password Valid
        API->>JWT: Generate Token (id, email, entity)
        JWT-->>API: JWT Token
        API->>Redis: Store Token (key: entity+id, value: token)
        API->>DB: Update Last Login
        API-->>Client: 200 OK + Token + User Data
    else Password Invalid
        API-->>Client: 401 Unauthorized
    end

    Note over Client,Bcrypt: Protected Route Access
    Client->>API: GET /protected-route (Bearer Token)
    API->>JWT: Verify Token Signature
    JWT-->>API: Decoded Token Data

    API->>Redis: Check Token Exists (entity+id)
    Redis-->>API: Token Found/Not Found

    alt Token Valid in Redis
        API->>DB: Find User by ID
        DB-->>API: User Record
        API->>API: Check User Status (active/blocked)

        alt User Active
            API->>API: Check Role Permissions
            alt Has Permission
                API->>DB: Fetch Requested Data
```

```
        DB-->>API: Data
        API-->>Client: 200 OK + Data
    else No Permission
        API-->>Client: 403 Forbidden
    end
else User Blocked
    API-->>Client: 401 User Blocked
end
else Token Invalid/Expired
    API-->>Client: 401 Unauthorized
end
```

```
Note over Client,Bcrypt: Logout Flow
Client->>API: POST /logout (with Token)
API->>JWT: Verify Token
JWT-->>API: Decoded Data
API->>Redis: Delete Token (entity+id)
Redis-->>API: Token Deleted
API-->>Client: 200 Logged Out
```



Email Notification Flow

Event-Driven Email System

flowchart TD

Event[System Event Triggered] --> EventType{Event Type?}

EventType -->|Order Created| OrderEvent[Order Created Event]

EventType -->|Order Dispatched| DispatchEvent[Order Dispatched Event]

EventType -->|Order Shipped| ShippedEvent[Order Shipped Event]

EventType -->|Invoice Sent| InvoiceEvent[Invoice Sent Event]

EventType -->|Payment Received| PaymentEvent[Payment Received Event]

EventType -->|User Registered| UserEvent[User Registered Event]

EventType -->|Account Approved| ApproveEvent[Account Approved Event]

EventType -->|Quotation Sent| QuotationEvent[Quotation Sent Event]

EventType -->|OTP Request| OTPEvent[OTP Request Event]

EventType -->|Employee Invite| InviteEvent[Employee Invite Event]

OrderEvent --> Email1[orderEmailtoCustomer]

OrderEvent --> Email2[orderEmailtoLocalPatner]

DispatchEvent --> Email3[orderDispatch]

ShippedEvent --> Email4[orderShipped]

InvoiceEvent --> Email5[sentInvoiceEmail]

PaymentEvent --> Email6[paidInvoiceEmail]

PaymentEvent --> Email7[paidInvoiceEmailAdminOrLocalPatner]

UserEvent --> Email8[userAccountCreated]

ApproveEvent --> Email9[userAccountApprove]

QuotationEvent --> Email10[coffeeMachineQuotation]

QuotationEvent --> Email11[leadQuotation]

OTPEvent --> Email12[otpToUsers]

OTPEvent --> Email13[otpToUsersForgotPassword]

InviteEvent --> Email14[inviteEmployee]

Email1 --> PrepareData[Prepare Email Data]

Email2 --> PrepareData

Email3 --> PrepareData

Email4 --> PrepareData

Email5 --> PrepareData

Email6 --> PrepareData

Email7 --> PrepareData

Email8 --> PrepareData

Email9 --> PrepareData

Email10 --> PrepareData

Email11 --> PrepareData

Email12 --> PrepareData

Email13 --> PrepareData

```
Email14 --> PrepareData
```

```
PrepareData --> RenderTemplate[Render EJS Template]
```

```
RenderTemplate --> AttachFiles{Attachments?}
```

```
AttachFiles -->|Yes| AttachPDF[Attach PDF/Images]
```

```
AttachFiles -->|No| SkipAttach[Skip Attachments]
```

```
AttachPDF --> SendEmail[Send via Nodemailer]
```

```
SkipAttach --> SendEmail
```

```
SendEmail --> SMTP[SMTP Server]
```

```
SMTP --> Recipient[Email Delivered]
```

```
Recipient --> LogEmail[Log Email Sent]
```

```
LogEmail --> Complete[Email Sent Successfully]
```



System Architecture Flow

Request Processing Pipeline

```
flowchart LR
    A[Client Request] --> B[Express Server]
    B --> C[Logging Middleware]
    C --> D[CORS Check]
    D --> E[Body Parser]
    E --> F[Helmet Security]
    F --> G[Rate Limiter]
    G --> H[Sanitize Input]
    H --> I{JWT Token?}

    I -->|Yes| J[Verify JWT]
    I -->|No| K[Public Route]

    J --> L[Check Redis Session]
    L --> M{Session Valid?}
    M -->|Yes| N[Load User Data]
    M -->|No| O[401 Unauthorized]

    N --> P[Role Check]
    P --> Q{Permission?}
    Q -->|Yes| R[Route Handler]
    Q -->|No| S[403 Forbidden]

    K --> R
    R --> T[Controller]
    T --> U[Service Layer]
    U --> V[Database Query]
    V --> W[Redis Cache Check]

    W --> X{Cached?}
    X -->|Yes| Y[Return Cached Data]
    X -->|No| Z[Query Database]

    Z --> AA[Process Data]
    AA --> AB[Cache Result]
    AB --> AC[Format Response]
    Y --> AC

    AC --> AD[Error Handler]
    AD --> AE[Response to Client]
```

O --> AD

S --> AD



Data Flow Summary

Complete System Data Flow

```
graph TB
    subgraph "Input Sources"
        A1[Customer Orders]
        A2[Admin Actions]
        A3[Sales Rep Actions]
        A4[Supplier Actions]
        A5[Webhooks]
    end

    subgraph "API Layer"
        B1[Express Routes]
        B2[Middleware]
        B3[Controllers]
    end

    subgraph "Business Logic"
        C1[Order Service]
        C2[Payment Service]
        C3[QBO Service]
        C4[Email Service]
        C5[Notification Service]
    end

    subgraph "Data Storage"
        D1[(MySQL Database)]
        D2[(Redis Cache)]
        D3[File Storage]
    end

    subgraph "External Services"
        E1[Stripe API]
        E2[QuickBooks Online]
        E3[Email SMTP]
        E4[Firebase FCM]
    end

    subgraph "Output"
        F1[Customer Notifications]
        F2[Admin Dashboard]
        F3[Reports]
        F4[Invoices]
    end
```

A1 --> B1
A2 --> B1
A3 --> B1
A4 --> B1
A5 --> B1

B1 --> B2
B2 --> B3
B3 --> C1
B3 --> C2
B3 --> C3
B3 --> C4
B3 --> C5

C1 --> D1
C2 --> D1
C3 --> D1
C4 --> D1
C5 --> D1

C1 --> D2
C2 --> D2
C3 --> D2

C4 --> D3

C2 --> E1
C3 --> E2
C4 --> E3
C5 --> E4

E1 --> C2
E2 --> C3
E3 --> C4
E4 --> C5

D1 --> F2
D1 --> F3
D3 --> F4
E4 --> F1
E3 --> F1



Diagram Legend

Relationship Types

- `||--o{` : One-to-Many (One entity has many related entities)
- `}o--||` : Many-to-One (Many entities belong to one entity)
- `||--||` : One-to-One
- `}o--o{` : Many-to-Many

Flow Chart Symbols

- `[ ]` : Process/Step
- `{ }` : Decision Point
- `( [ ] )` : Start/End Point
- `-->` : Flow Direction

Sequence Diagram

- `-->>` : Synchronous call
- `-->>` : Response/Return
- `Note over` : Annotation

End of ERD and Flow Diagrams

These diagrams provide a comprehensive visual representation of the Busy Beans backend system architecture, database relationships, and key business workflows.